

Arbres de Décision

Octobre 2026

Cédric Dangeard

cedric.dangeard@orange.com



Business

Cours 2 : Au programme

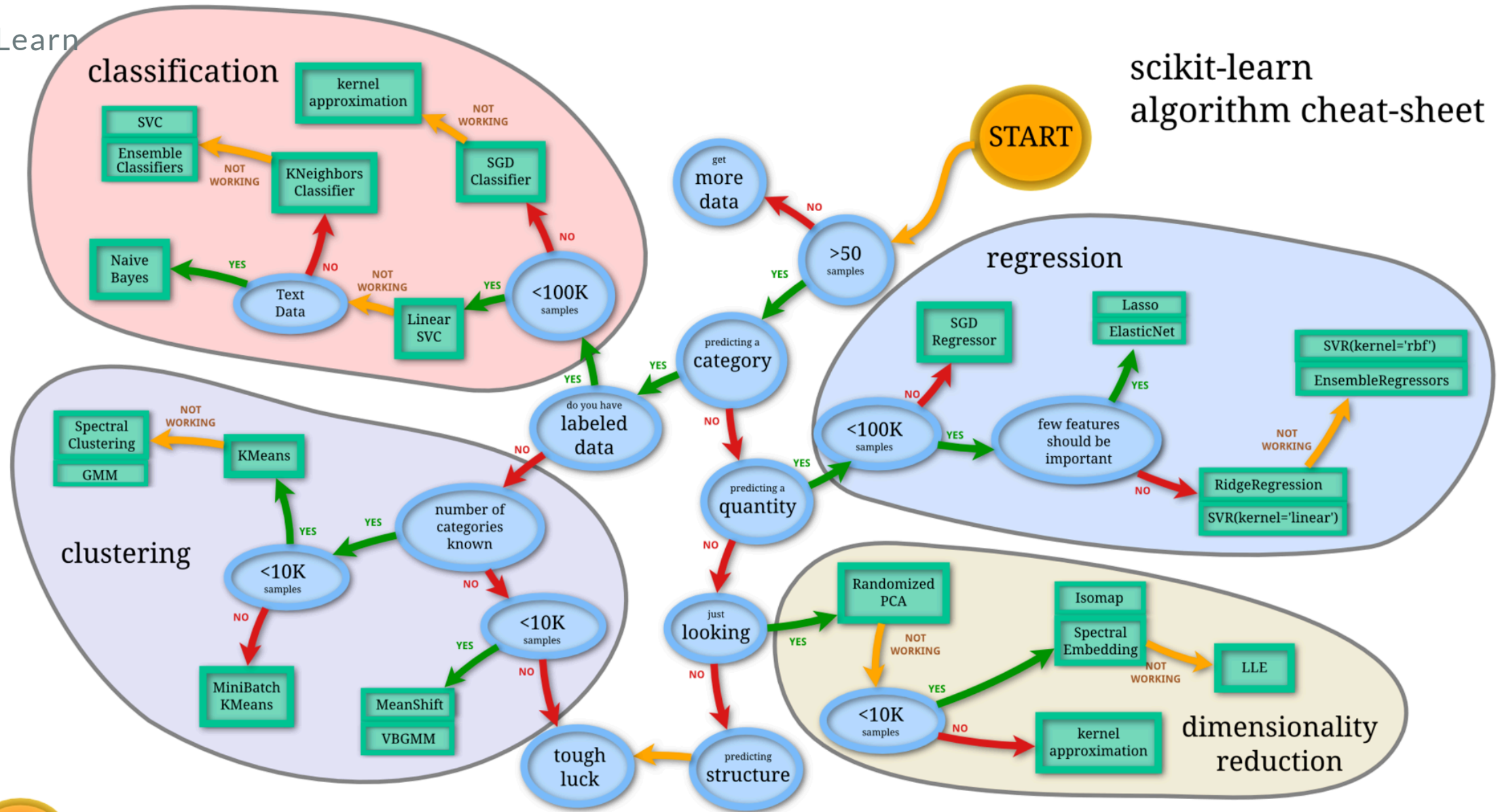
- Les bibliothèques fondamentales
- Nettoyage et préparation des données
 - Gestion des valeurs manquantes
 - Encodage des variables catégorielles
 - Échantillonnage des données
- TP : Nettoyage des données
- Les arbres de décision
- TP : Implémentation d'un arbre

Les bibliothèques fondamentales

- Numpy
 - *alias* : Numerical Python
 - *objet de base* : `numpy.array`
 - *missions* : puissance et rapidité de calcul sur des tableaux multidimensionnels
 - *implémentation* : C
 - [code](#), [site](#)
- Scipy
 - *alias* : Scientific Python
 - *objet de base* : `numpy`
 - *missions* : algorithmes de plus haut niveau (optimisation, régression, interpolation, équations différentielles, ...)
 - *implémentation* : C, Fortran, C++, Cython

Scikit-Learn

- Bibliothèque de machine learning en Python
- Libre et open source
- Orientée objets
- Documentée
- Communauté active
- [code](#), [site](#)



- Estimateur
 - fit : Entraîner le modèle
 - transform : Transformer les données
 - predict : Prédire la variable cible
 - score : Évaluer les performances du modèle

```
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin

class MyEstimator(BaseEstimator, TransformerMixin):
    def __init__(self, param: int = 1):
        self.param = param

    def fit(self, X, y=None):
        self.is_fitted_ = True
        return self

    def transform(self, X):
        return X

    def predict(self, X):
        return np.full(shape=X.shape[0], fill_value=self.param)
```

Encodage des variables catégorielles

Pourquoi encoder les variables catégorielles ?

- La plupart des algorithmes de machine learning ne peuvent pas travailler directement avec des données non numériques.
- Les variables catégorielles doivent être converties en une représentation numérique pour être utilisées comme entrées dans les modèles.
- Le choix dépend du modèle, du volume de données et de la cardinalité des variables.

Label Encoding

- Chaque modalité prend une valeur
- Peut être ordinal
- Pas de nouvelles variables
- Peut introduire un ordre artificiel
- Que faire des valeurs manquantes ?

```
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder

df = pd.DataFrame({
    'Name' : ['Sweet Mask', 'Bald Cape', 'Blizzard of Hell',
             'King', 'Glasses', 'Metal Bat', 'Mumen Rider'],
    'Class' : ['A', 'C', 'B', 'S', 'A', 'S', 'C']})

le = LabelEncoder()
oe = OrdinalEncoder(categories=[['S', 'A', 'B', 'C']])

df['label'] = le.fit_transform(df['Class'])
df['ord'] = oe.fit_transform(df['Class'])
```

- **LabelEncoder** est principalement conçu pour encoder la cible **y**.

Exemple label encoding

Name	Sweet Mask	Bald Cape	Blizzard of Hell	King	Glasses	Metal Bat	Mumen Rider		
Class	A	C	B	S	A	S	C		
label	0	2	1	3	0	3	2		
ord	1.0	3.0	2.0	0.0	1.0	0.0	3.0		

One Hot Encoding

- Une nouvelle variable par modalité
- Chaque variable prend la valeur 0 ou 1
- Peut ajouter beaucoup de variables
- Deux façons de faire :
 - `pandas.get_dummies`
 - `sklearn.OneHotEncoder`

```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(categories=[['S', 'A', 'B', 'C']],
                    drop = 'first',
                    sparse_output=False)

hot_encoded = ohe.fit_transform(df[['Class']])

data_hot_encoded = pd.DataFrame(hot_encoded, index=df.index)
data_hot_encoded.columns = ohe.get_feature_names_out()

pd.concat([df, data_hot_encoded], axis=1)
```

```
pd.concat([df,
          pd.get_dummies(df[['Class']], drop_first=True)], axis=1)
```

Name	Sweet Mask	Bald Cape	Blizzard of Hell	King	Glasses	Metal Bat	Mumen Rider
Class	A	C	B	S	A	S	C
Class_A	1.0	0.0	0.0	0.0	1.0	0.0	0.0
Class_B	0.0	0.0	1.0	0.0	0.0	0.0	0.0
Class_C	0.0	1.0	0.0	0.0	0.0	0.0	1.0

Target Encoding

- Remplacer les modalités par une valeur dérivée de la cible
 - Peut créer du sur-apprentissage
 - À utiliser sur des variables de forte cardinalité
 - À appliquer uniquement sur l'échantillon d'entraînement (risque de fuite de données)

```
df = pd.DataFrame({
    'Name' : ['LeBron', 'Kawamura', 'Wembanyama', 'Durant',
             'Curry', 'Jokić', 'Paul', 'Davis'],
    'Taille' : [206, 173, 224, 211, 188, 211, 183, 208],
    'Club' : ['Lakers', 'Grizzlies', 'Spurs', 'Suns',
             'Warriors', 'Nuggets', 'Spurs', 'Lakers',]
})

df['encoded_Club'] = df.groupby('Club')['Taille'].transform('mean')
```

Name	LeBron	Kawamura	Wembanyama	Durant	Curry	Jokić	Paul	Davis
Taille	206	173	224	211	188	211	183	208
Club	Lakers	Grizzlies	Spurs	Suns	Warriors	Nuggets	Spurs	Lakers
encoded_Club	207.0	173.0	203.5	211.0	188.0	211.0	203.5	207.0

```
from sklearn.preprocessing import TargetEncoder

te = TargetEncoder(smooth='auto',
                   cv=2)

target_encoded = te.fit_transform(X=df[['Club']], y = df[['Taille']])
```

Échantillonnage des données

- **Sur-échantillonnage aléatoire** : dupliquer des observations de la classe minoritaire. Conserve toutes les données, mais peut favoriser le sur-apprentissage.
- **Sous-échantillonnage aléatoire** : retirer des observations de la classe majoritaire. Réduit la taille du jeu de données, mais peut perdre de l'information.
- [Imbalanced-learn](#) : bibliothèque Python pour le rééchantillonnage

Méthodes de rééchantillonnage

- **Données synthétiques** : créer de nouvelles observations minoritaires à partir d'observations proches.
 - **SMOTE** : interpole entre voisins de la classe minoritaire.
 - **ADASYN** : génère davantage d'observations dans les zones où la classe minoritaire est difficile à distinguer.

Attention : séparer entraînement et test avant le rééchantillonnage ; ne rééquilibrer que les données d'entraînement.

TP : Encodage

Des Questions
avant de s'y mettre?



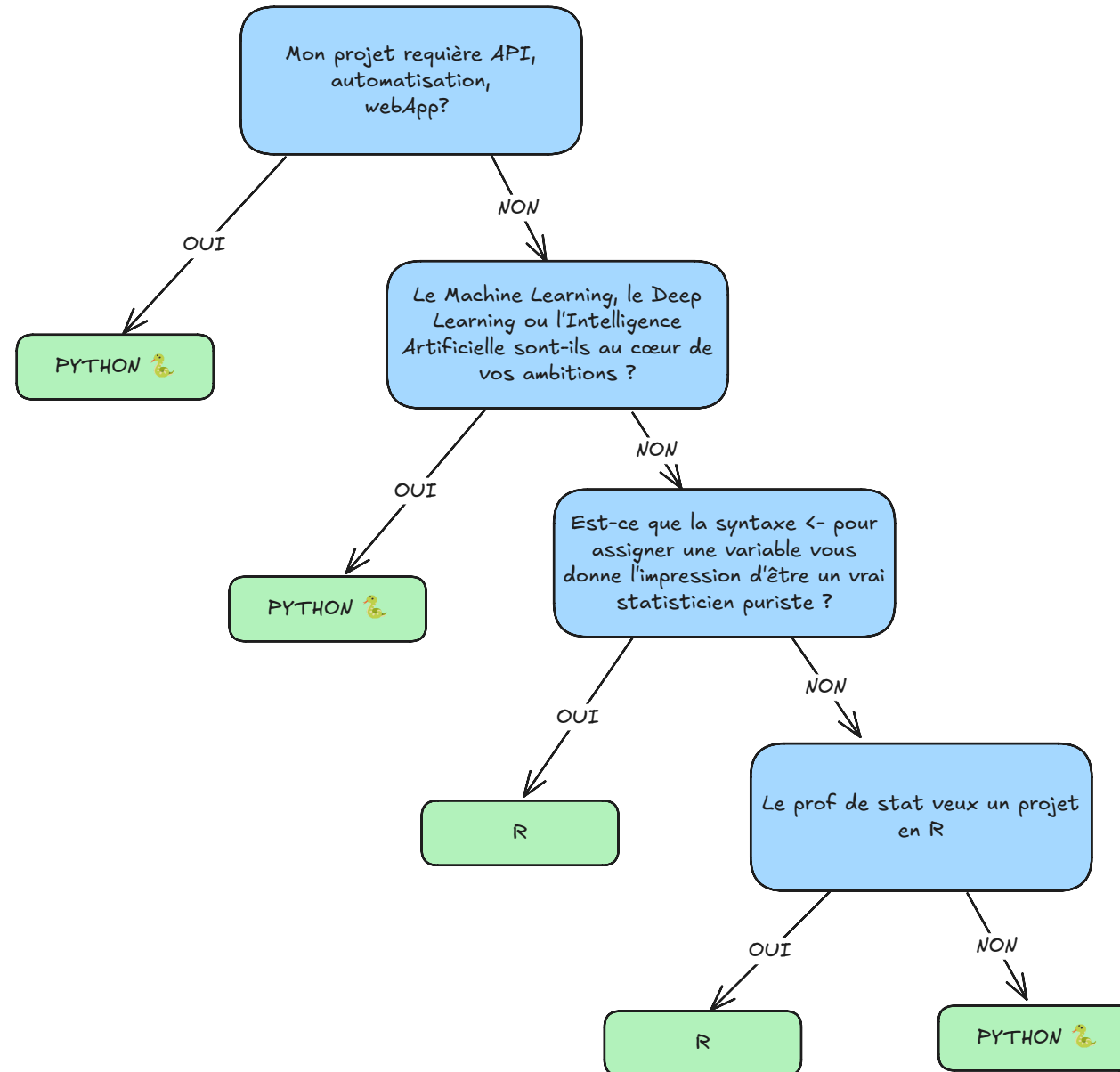
Arbres de décision

Qu'est-ce qu'un arbre de décision ?

Un arbre de décision est un modèle supervisé qui applique une suite de règles de décision pour prédire une classe (classification) ou une valeur (régression).

Vocabulaire sur les arbres

- **Nœud racine** : Le nœud supérieur de l'arbre, qui ne possède pas de parent.
- **Nœud de décision** : Un nœud à partir duquel on divise les données selon une caractéristique.
- **Feuille** (ou **nœud terminal**) : Un nœud qui ne se divise pas davantage et représente une prédiction ou une classe.
- **Branche** : Une connexion entre deux nœuds, représentant le flux de décision.



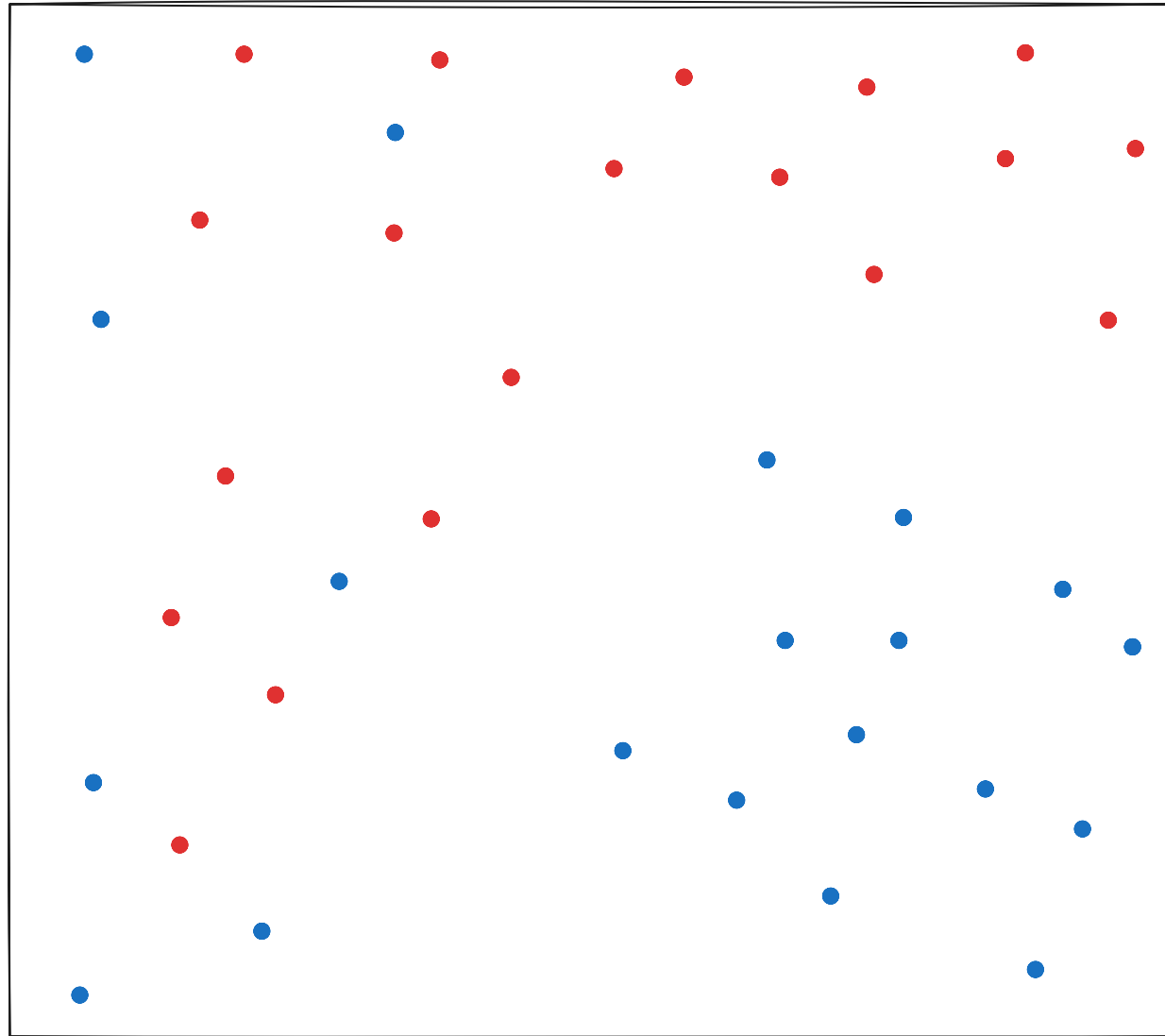
Arbre de décision

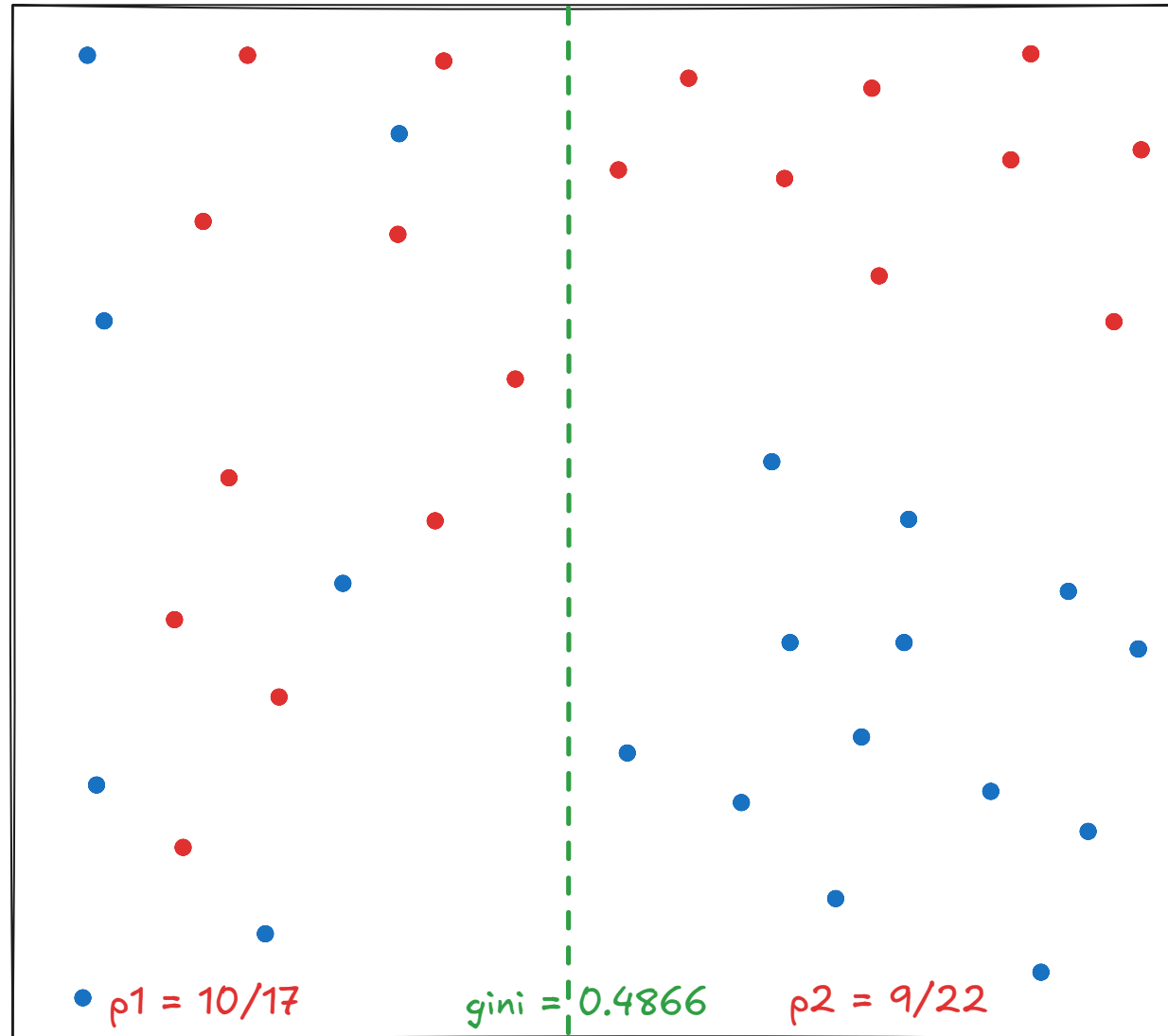
- Modèle de classification et de régression
- Structure arborescente composée de nœuds
- Chaque nœud représente une décision basée sur une caractéristique
- Chaque branche représente le résultat d'une décision
- Chaque feuille représente une prédiction ou une classe

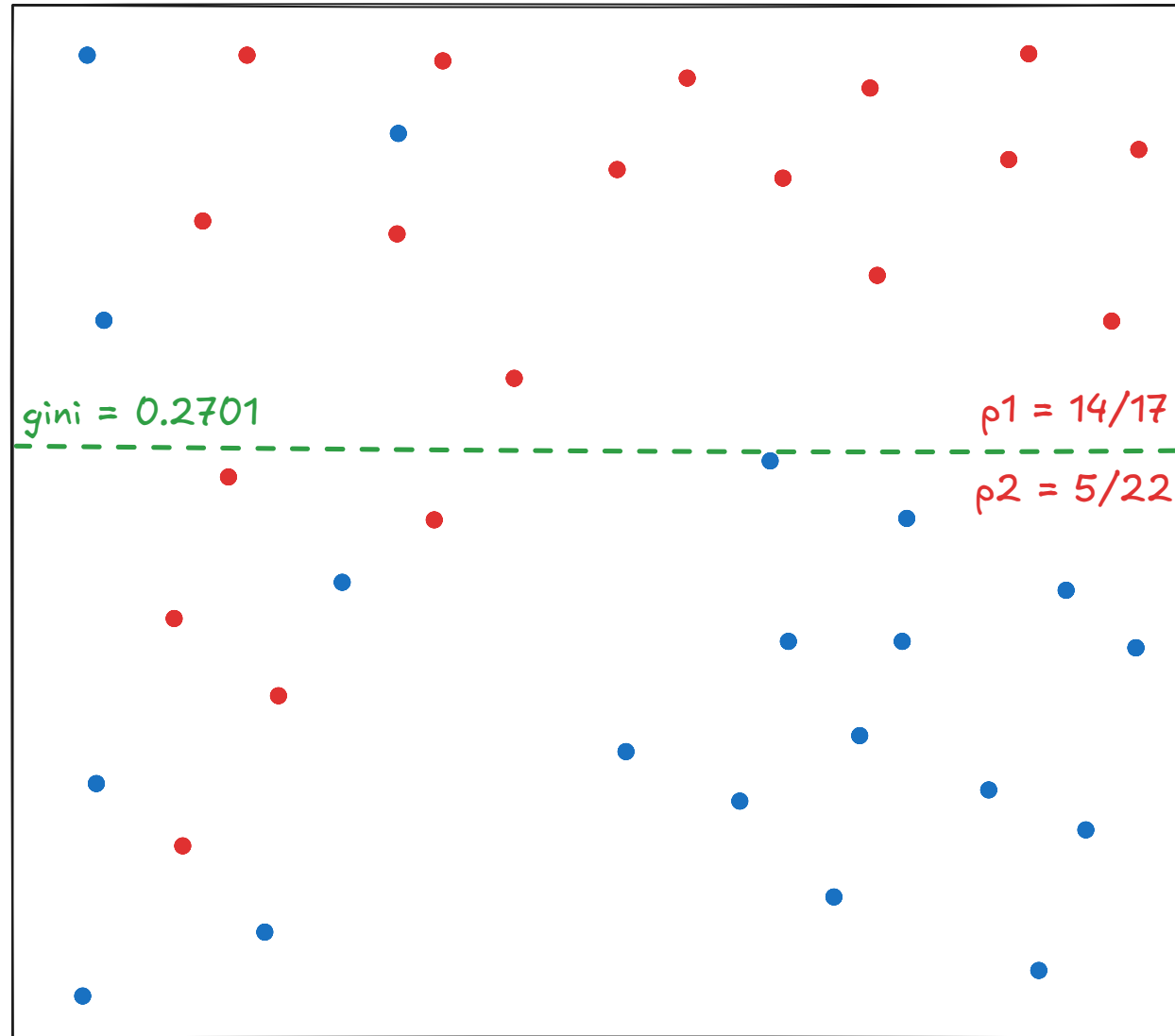
Construction d'un arbre de décision

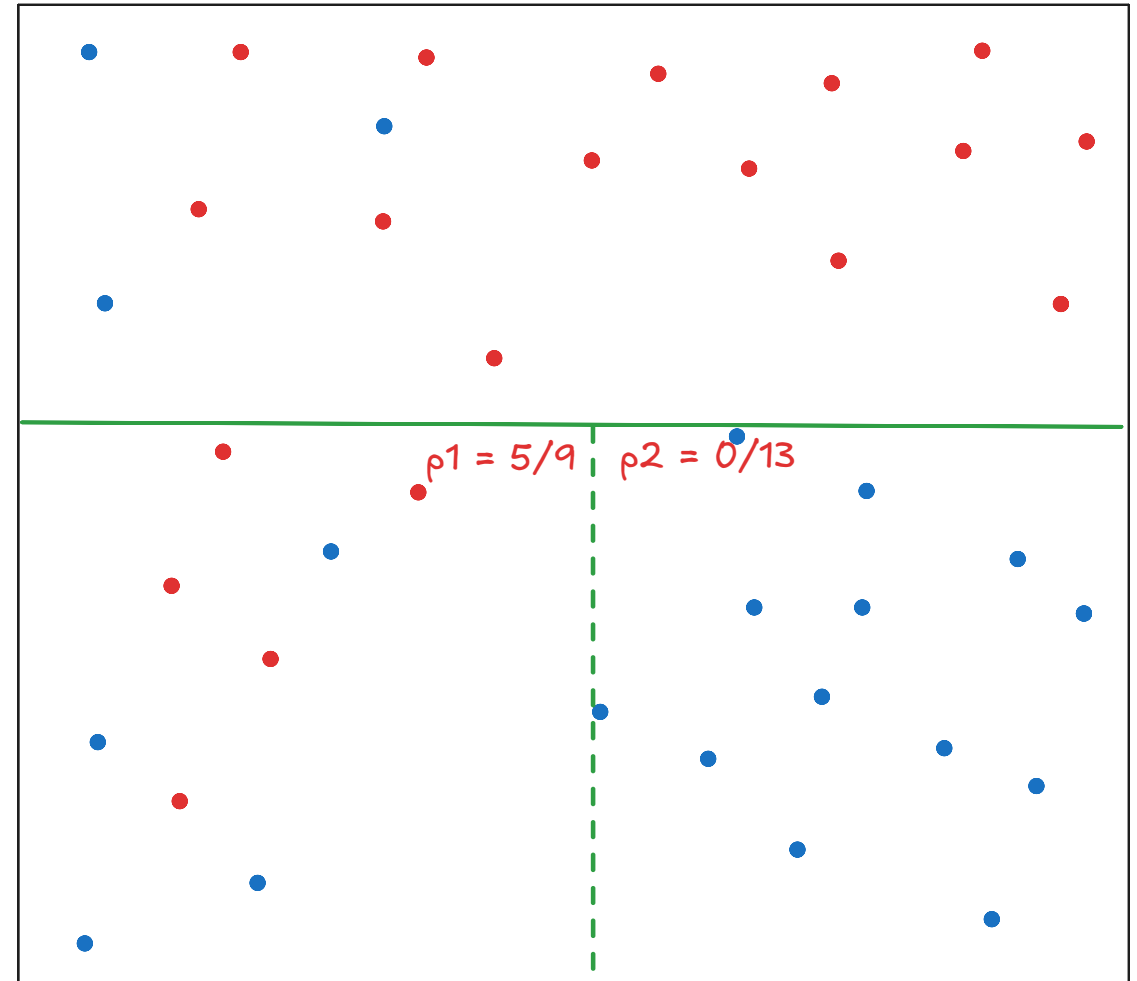
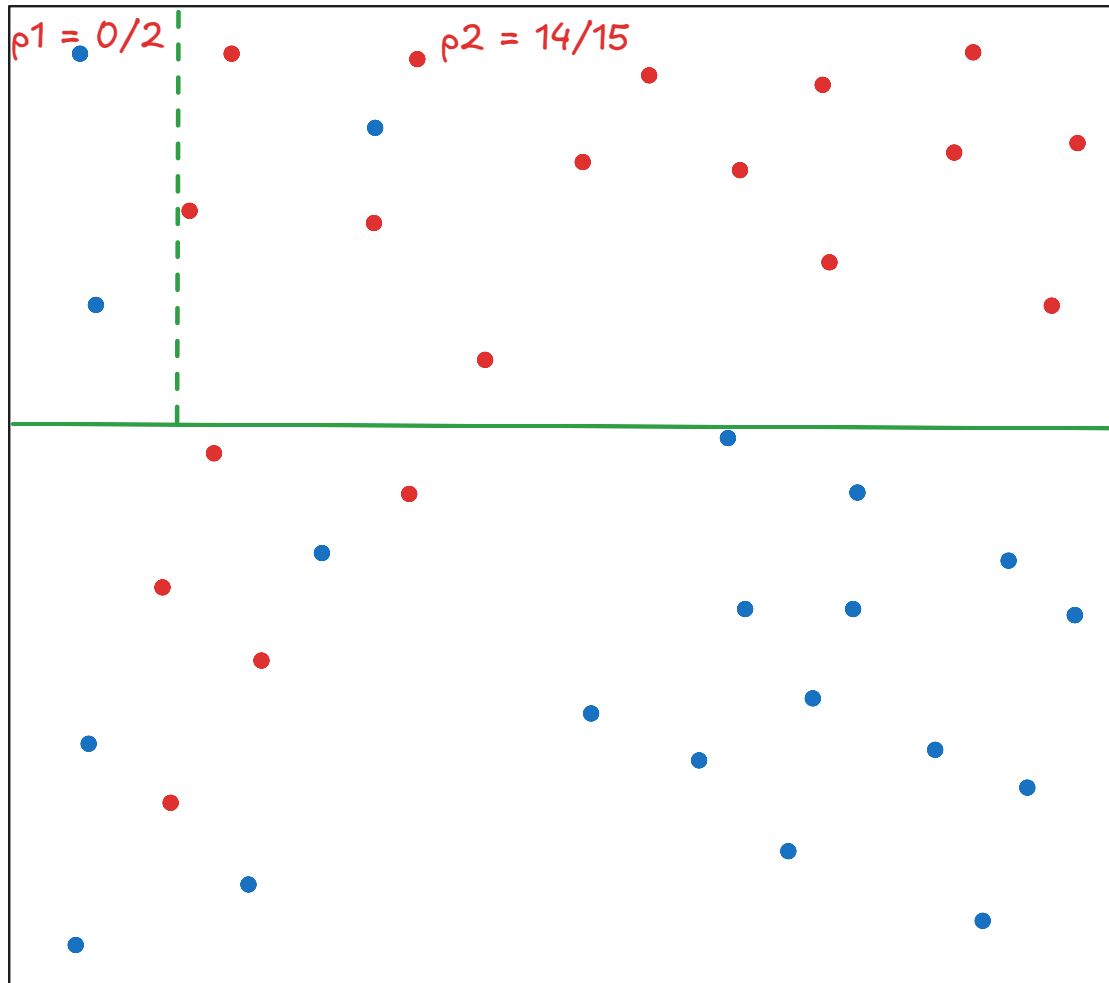
La construction se fait par *partitionnement récursif* des données.

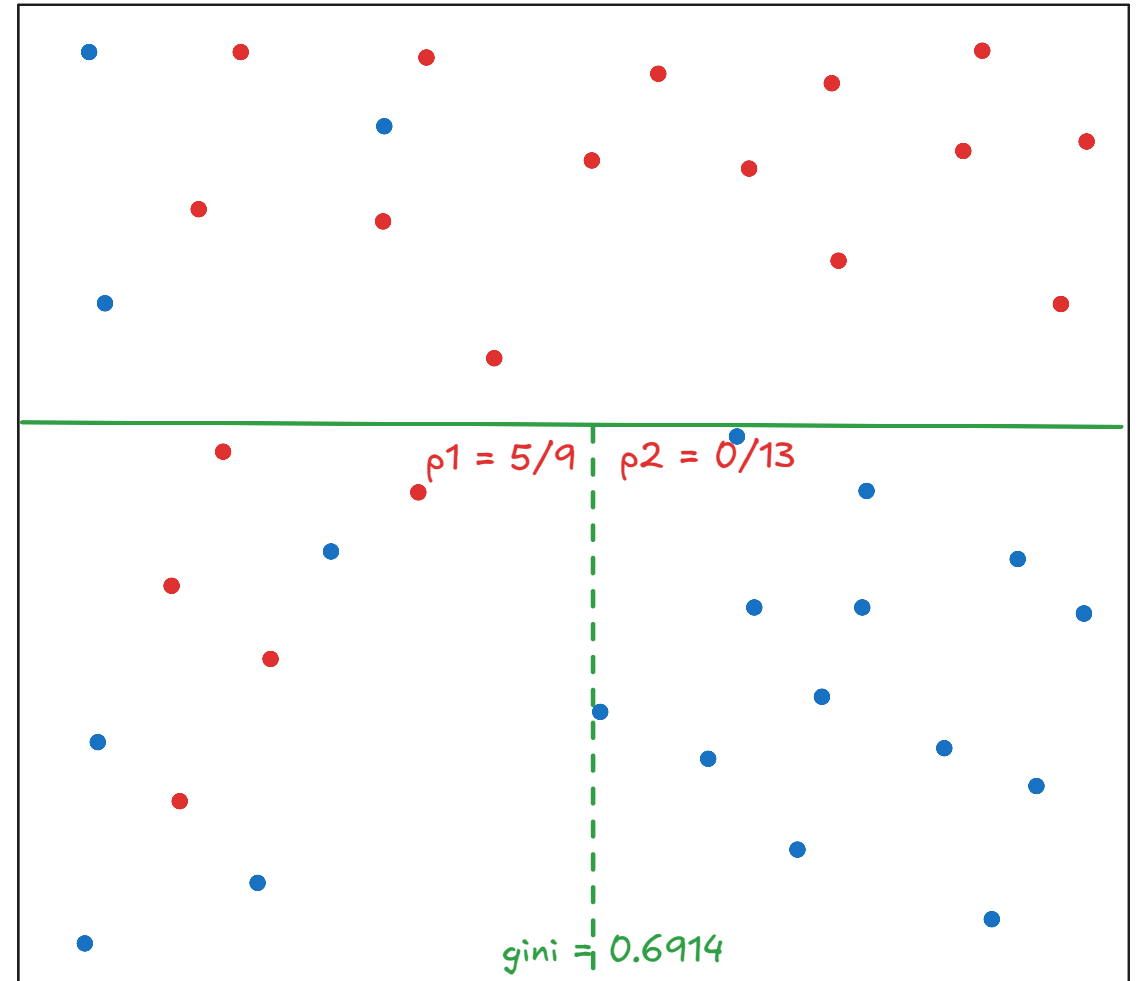
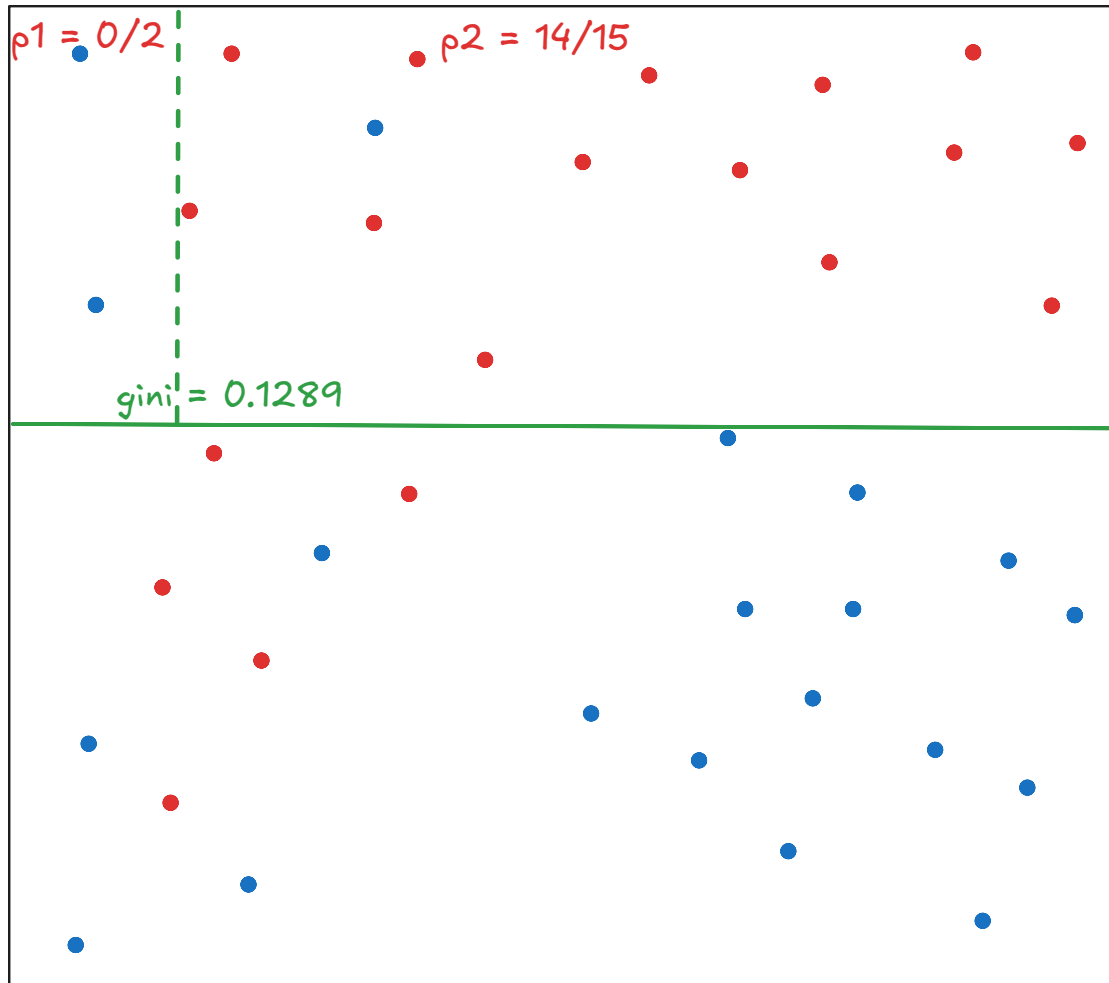
1. Choisir la meilleure division pour diviser les données.
2. Diviser les données en sous-ensembles basés sur cette caractéristique.
3. Répéter le processus pour chaque sous-ensemble jusqu'à ce qu'un critère d'arrêt soit atteint

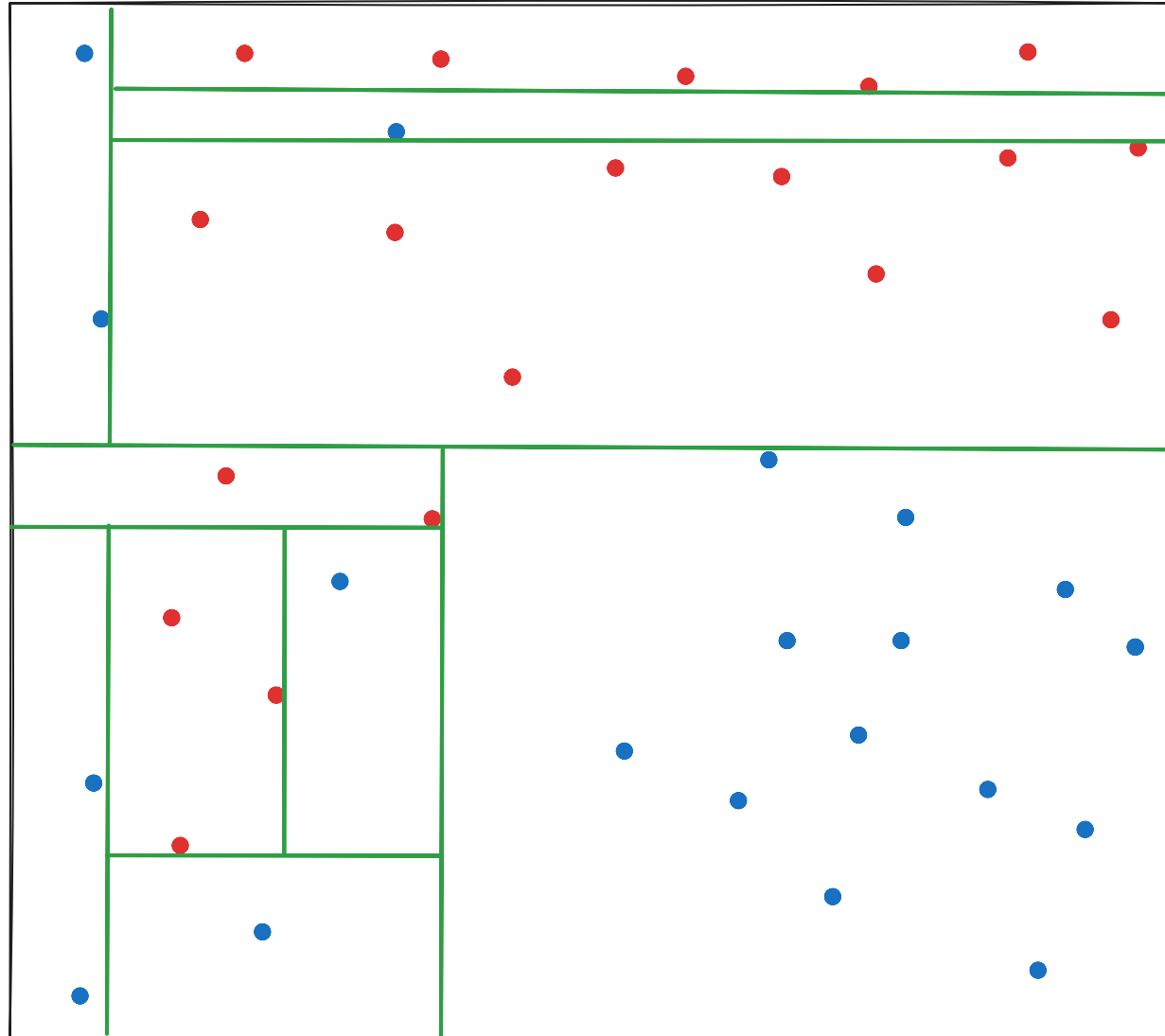




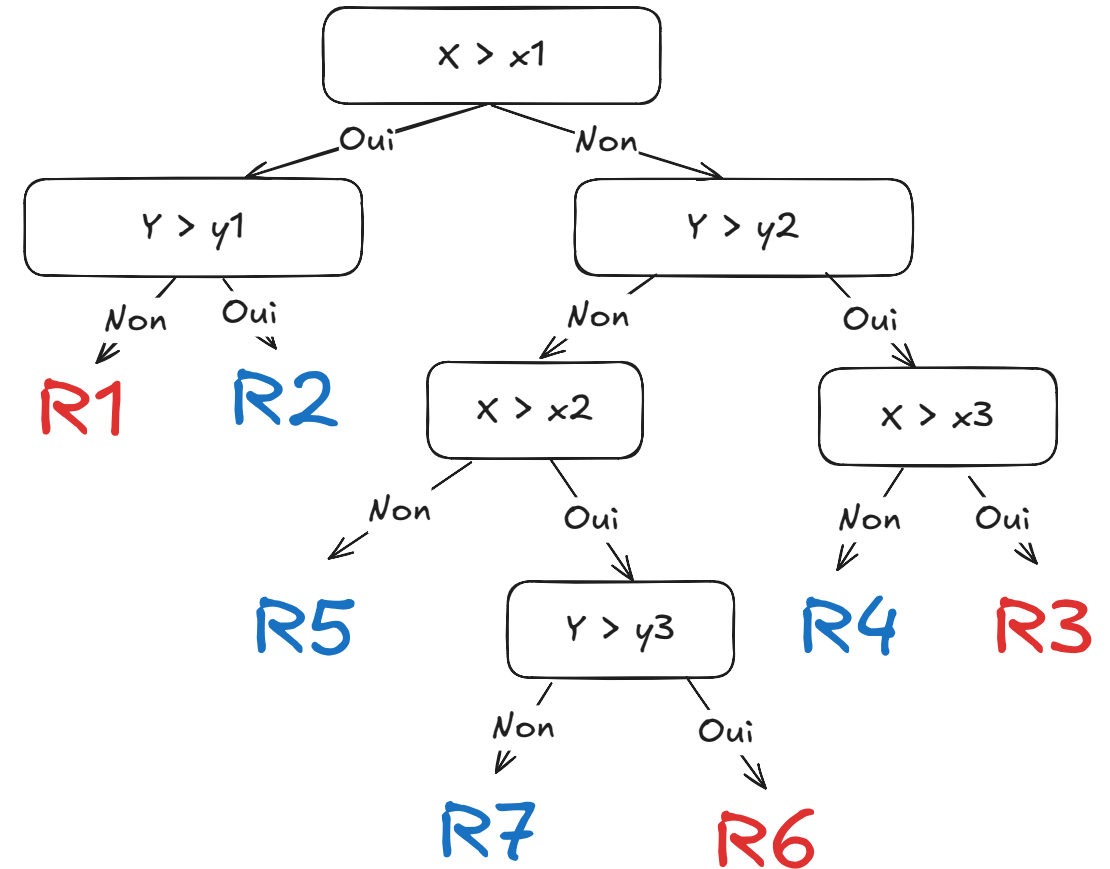
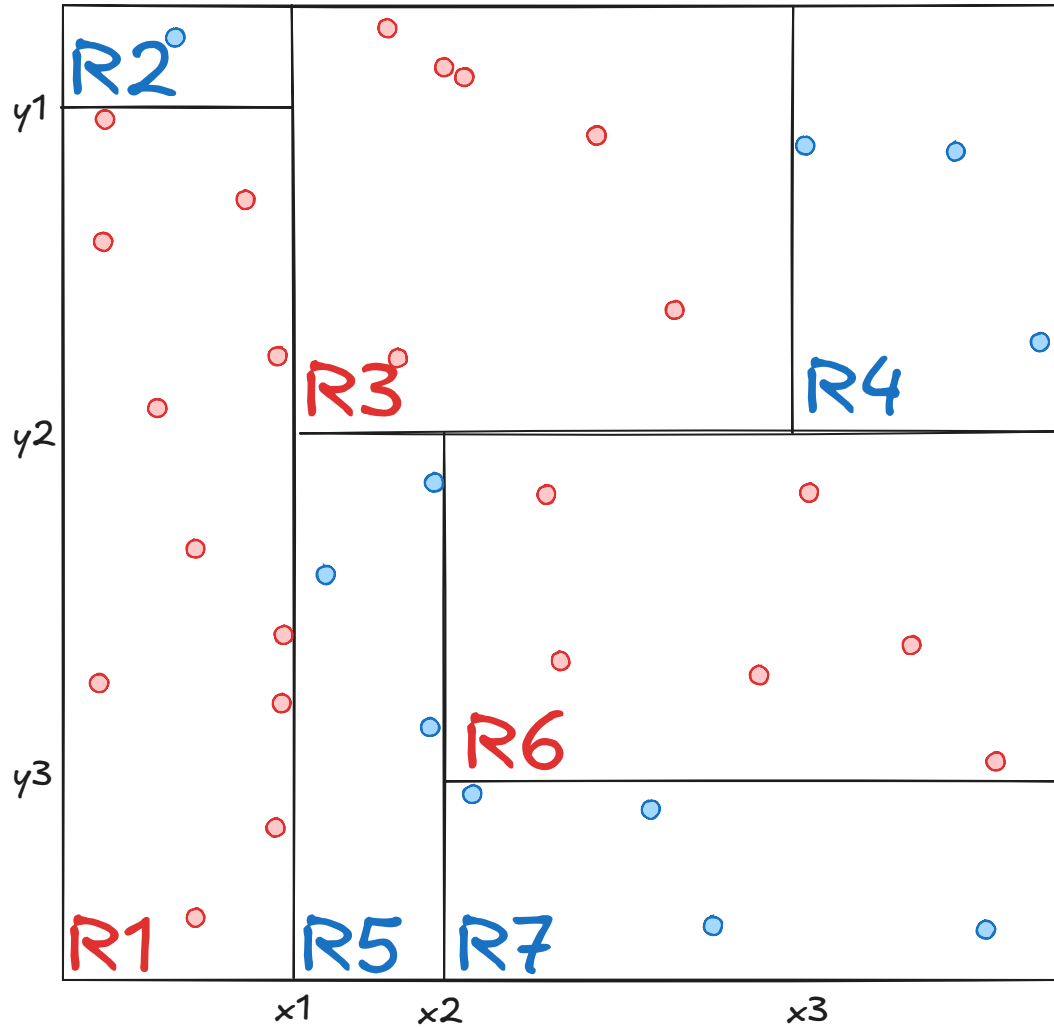








Arbres de décision



Comment choisir la meilleure division ?

- Quelle variable choisir ?
- Quel seuil de division ?

CART : L'algorithme

- Définir un critère d'impureté
 - CART : indice de Gini : $I_G(f) = \sum_{i=1}^m f_i(1 - f_i) = 1 - \sum_{i=1}^m f_i^2$
où f_i est la proportion de classe i dans le nœud considéré.
 - Autre critère d'impureté : [entropie de Shannon](#).
- Pour chaque division candidate, calculer la moyenne pondérée des impuretés des nœuds enfants.
- Choisir la division qui minimise cette impureté (ou maximise la réduction d'impureté).

CART : Exemple de division

- Nœud parent : 8 observations, 4 de classe A et 4 de classe B.
 $I_G(\text{parent}) = 1 - (4/8)^2 - (4/8)^2 = 0,5$
- Division candidate : à gauche 3 A et 1 B ; à droite 1 A et 3 B.
Chaque nœud enfant a une impureté de
 $1 - (3/4)^2 - (1/4)^2 = 0,375$.
- Impureté après division : $\frac{4}{8} I_G(\text{gauche}) + \frac{4}{8} I_G(\text{droite}) = 0,375$.
- **Réduction d'impureté** : $0,5 - 0,375 = 0,125$.

On compare cette réduction à celle des autres divisions possibles et on retient la plus élevée.

CART : Où s'arrêter ?

Un individu par feuille ?

Jusqu'à avoir des feuilles seules classes ?

Taille des feuilles, nombre de feuilles ?

Profondeur de l'arbre ?

CART : Où s'arrêter ? Que cherche-t-on à éviter ?

- Un individu par feuille ?
- Une classe par feuille ?
- Taille des feuilles, nombre de feuilles ?
- Profondeur de l'arbre ?

Le choix de ce critère d'arrêt constitue l'un des hyperparamètres à optimiser de notre modèle.

CART : Pruning

Plus un arbre est profond, plus la variance est élevée, et le biais est faible.

- Élagage (*pruning*)
 - Pré-élagage :
 - Instaurer des règles d'arrêt pendant l'apprentissage
 - Post-élagage :
 - Partir d'un arbre profond et réduire la variance en supprimant des feuilles.

Exemple d'élagage avec ccp_alpha

```
from sklearn.tree import DecisionTreeClassifier

# Création d'un arbre de décision avec ccp_alpha
tree = DecisionTreeClassifier(ccp_alpha=0.01)
tree.fit(X_train, y_train)
```

La valeur de `ccp_alpha` contrôle la sévérité de l'élagage. Une valeur plus élevée entraîne un arbre plus petit et moins complexe.

Arbres de décision : Conclusion

- Interprétable
- Variance élevée
- Sur-apprentissage pour les arbres trop profonds
- Performance limitée sur des ensembles de données complexes

Solutions :

- Comment améliorer les performances des arbres ?
- Comment réduire la variance ?

CART sur scikit-learn : DecisionTreeClassifier

- Paramètres :
 - **criterion**
 - **max_depth**
 - **min_samples_split**
 - **min_samples_leaf**
 - **max_features**

CART sur scikit-learn : DecisionTreeClassifier

- Paramètres :
 - **criterion** [*gini, entropy, log_loss*]: critères d'homogénéité
 - **max_depth** *int*: Profondeur de l'arbre
 - **min_samples_split** *int*: nombre minimal d'individus dans un nœud pour tenter une division
 - **min_samples_leaf** *int*: nombre minimal d'individus dans chaque feuille fille pour accepter la division
 - **max_features** [*int, float, 'sqrt', 'log2', None*]: nombre maximal de variables testées pour créer un nœud

TP : Arbres de décision